



# SMART CONTRACT AUDIT REPORT

for

## AllDeFi Vault



Prepared By: Xiaomi Huang

PeckShield  
June 24, 2026

## Document Properties

|                |                             |
|----------------|-----------------------------|
| Client         | AllDeFi                     |
| Title          | Smart Contract Audit Report |
| Target         | AllDeFi                     |
| Version        | 1.0                         |
| Author         | Xuxian Jiang                |
| Auditors       | Matthew Jiang, Xuxian Jiang |
| Reviewed by    | Xiaomi Huang                |
| Approved by    | Xuxian Jiang                |
| Classification | Public                      |

## Version Info

| Version | Date          | Author(s)    | Description          |
|---------|---------------|--------------|----------------------|
| 1.0     | June 24, 2026 | Xuxian Jiang | Final Release        |
| 1.0-rc  | June 22, 2026 | Xuxian Jiang | Release Candidate #1 |

## Contact

For more information about this document and its contents, please contact PeckShield Inc.

|       |                        |
|-------|------------------------|
| Name  | Xiaomi Huang           |
| Email | contact@peckshield.com |

## Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>4</b>  |
| 1.1      | About AllDeFi Vault . . . . .                                   | 4         |
| 1.2      | About PeckShield . . . . .                                      | 5         |
| 1.3      | Methodology . . . . .                                           | 5         |
| 1.4      | Disclaimer . . . . .                                            | 7         |
| <b>2</b> | <b>Findings</b>                                                 | <b>9</b>  |
| 2.1      | Summary . . . . .                                               | 9         |
| 2.2      | Key Findings . . . . .                                          | 10        |
| <b>3</b> | <b>Detailed Results</b>                                         | <b>11</b> |
| 3.1      | Inconsistent Pause Enforcement Among Claim* Functions . . . . . | 11        |
| 3.2      | Trust Issue of Admin Keys . . . . .                             | 12        |
| <b>4</b> | <b>Conclusion</b>                                               | <b>15</b> |
|          | <b>References</b>                                               | <b>16</b> |

# 1 | Introduction

Given the opportunity to review the design document and related source code of the `AllDeFi Vault` protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

## 1.1 About AllDeFi Vault

`AllDeFi` is a vault infrastructure protocol for professionally managed `DeFi` strategies with institutional custody. It offers a structured vault framework for users to access curated yield opportunities. The audited `vault` facilitates user investment through an epoch-based structure, with deposits processed in a controlled and secure manner. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of AllDeFi

| Item                | Description    |
|---------------------|----------------|
| Name                | AllDeFi        |
| Type                | Smart Contract |
| Language            | Solidity       |
| Audit Method        | Whitebox       |
| Latest Audit Report | June 24, 2026  |

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- <https://github.com/alldefi/eth-vault.git> (af63fc6)

And here is the commit hash value after all fixes for the issues found in the audit have been applied or confirmed:

- <https://github.com/alldefi/eth-vault.git> (a6afd02)

## 1.2 About PeckShield

PeckShield Inc. [7] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email ([contact@peckshield.com](mailto:contact@peckshield.com)).

Table 1.2: Vulnerability Severity Classification

|        |        |            |        |        |
|--------|--------|------------|--------|--------|
| Impact | High   | Critical   | High   | Medium |
|        | Medium | High       | Medium | Low    |
|        | Low    | Medium     | Low    | Low    |
|        |        | High       | Medium | Low    |
|        |        | Likelihood |        |        |

## 1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [6]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact, and can be accordingly classified into four categories, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further

Table 1.3: The Full List of Check Items

| Category                    | Check Item                                |
|-----------------------------|-------------------------------------------|
| Basic Coding Bugs           | Constructor Mismatch                      |
|                             | Ownership Takeover                        |
|                             | Redundant Fallback Function               |
|                             | Overflows & Underflows                    |
|                             | Reentrancy                                |
|                             | Money-Giving Bug                          |
|                             | Blackhole                                 |
|                             | Unauthorized Self-Destruct                |
|                             | Revert DoS                                |
|                             | Unchecked External Call                   |
|                             | Gasless Send                              |
|                             | Send Instead Of Transfer                  |
|                             | Costly Loop                               |
|                             | (Unsafe) Use Of Untrusted Libraries       |
|                             | (Unsafe) Use Of Predictable Variables     |
|                             | Transaction Ordering Dependence           |
|                             | Deprecated Uses                           |
| Semantic Consistency Checks | Semantic Consistency Checks               |
| Advanced DeFi Scrutiny      | Business Logics Review                    |
|                             | Functionality Checks                      |
|                             | Authentication Management                 |
|                             | Access Control & Authorization            |
|                             | Oracle Security                           |
|                             | Digital Asset Escrow                      |
|                             | Kill-Switch Mechanism                     |
|                             | Operation Trails & Event Generation       |
|                             | ERC20 Idiosyncrasies Handling             |
|                             | Frontend-Contract Integration             |
|                             | Deployment Consistency                    |
|                             | Holistic Risk Management                  |
| Additional Recommendations  | Avoiding Use of Variadic Byte Array       |
|                             | Using Fixed Compiler Version              |
|                             | Making Visibility Level Explicit          |
|                             | Making Type Inference Explicit            |
|                             | Adhering To Function Declaration Strictly |
|                             | Following Other Best Practices            |

deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [5], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

## 1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

| Category                                             | Summary                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Configuration</b>                                 | Weaknesses in this category are typically introduced during the configuration of the software.                                                                                                                                                                                                              |
| <b>Data Processing Issues</b>                        | Weaknesses in this category are typically found in functionality that processes data.                                                                                                                                                                                                                       |
| <b>Numeric Errors</b>                                | Weaknesses in this category are related to improper calculation or conversion of numbers.                                                                                                                                                                                                                   |
| <b>Security Features</b>                             | Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)                                                                                                           |
| <b>Time and State</b>                                | Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.                                                                                              |
| <b>Error Conditions, Return Values, Status Codes</b> | Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.                                                                     |
| <b>Resource Management</b>                           | Weaknesses in this category are related to improper management of system resources.                                                                                                                                                                                                                         |
| <b>Behavioral Issues</b>                             | Weaknesses in this category are related to unexpected behaviors from code that an application uses.                                                                                                                                                                                                         |
| <b>Business Logics</b>                               | Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.                                                                                |
| <b>Initialization and Cleanup</b>                    | Weaknesses in this category occur in behaviors that are used for initialization and breakdown.                                                                                                                                                                                                              |
| <b>Arguments and Parameters</b>                      | Weaknesses in this category are related to improper use of arguments or parameters within function calls.                                                                                                                                                                                                   |
| <b>Expression Issues</b>                             | Weaknesses in this category are related to incorrectly written expressions within code.                                                                                                                                                                                                                     |
| <b>Coding Practices</b>                              | Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained. |



## 2 | Findings

### 2.1 Summary

Here is a summary of our findings after analyzing the implementation of the AllDeFi Vault protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

| Severity      | # of Findings |                                                                                     |
|---------------|---------------|-------------------------------------------------------------------------------------|
| Critical      | 0             |                                                                                     |
| High          | 0             |                                                                                     |
| Medium        | 1             |  |
| Low           | 0             |                                                                                     |
| Informational | 1             |  |
| Total         | 2             |                                                                                     |

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

## 2.2 Key Findings

Overall, this smart contract is well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability and 1 informational suggestion.

Table 2.1: Key AllDeFi Audit Findings

| ID      | Severity      | Title                                                                 | Category          | Status    |
|---------|---------------|-----------------------------------------------------------------------|-------------------|-----------|
| PVE-001 | Informational | <a href="#">Inconsistent Pause Enforcement Among Claim* Functions</a> | Business Logic    | Resolved  |
| PVE-002 | Medium        | <a href="#">Trust Issue of Admin Keys</a>                             | Security Features | Mitigated |

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contract is being deployed on mainnet. Please refer to Section 3 for details.



## 3 | Detailed Results

### 3.1 Inconsistent Pause Enforcement Among Claim\* Functions

- ID: PVE-001
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: Vault
- Category: Coding Practices [4]
- CWE subcategory: CWE-1126 [1]

#### Description

The audited `vault` contract implements an epoch-based vault with internal, non-transferable share accounting. It also inherits from the `PausableUpgradeable` contract so that the authorized admin account may pause the vault so that existing claims may be administratively blocked. While reviewing current claim logic, we notice the pause flag is not consistently enforced.

In the following, we show the implementation of three claim-related functions, i.e., `claim()`, `claimEth()`, and `claimFor()`. We notice the first two claim functions have the `whenNotPaused` modifier while the last `claimFor()` function does not have the `whenNotPaused` modifier. As a result, the last claim function does not honor the pause flag. For consistency, it is better to consistently enforce the pause parameter among all these claim functions.

```
297  /**
298   * @dev Claim asset tokens after settlement.
299   */
300  function claim() external whenNotPaused nonReentrant {
301      _claim(msg.sender, false);
302  }

304  /**
305   * @dev Claim WETH redemption assets as ETH.
306   * @notice Reverts if the recipient cannot receive ETH.
307   */
308  function claimEth() external whenNotPaused nonReentrant {
309      _claim(msg.sender, true);
```

310     }

Listing 3.1: Vault::claim()/claimEth()

```

391     /**
392      * @dev Claims settled redemption assets for an account, callable by the settler.
393      */
394     function claimFor(address account) external onlyRole(SETTLER_ROLE) nonReentrant {
395         if (account == address(0)) revert ZeroAddress();
396         _claim(account, false);
397     }

```

Listing 3.2: Vault::claim()/claimEth()

**Recommendation** Revisit the above claim functions to ensure the pause flag is consistently honored.

**Status** The issue has been resolved by the following commit: a6afd02.

## 3.2 Trust Issue of Admin Keys

- ID: PVE-002
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: Vault
- Category: Security Features [3]
- CWE subcategory: CWE-287 [2]

### Description

The AllDeFi Vault protocol is designed with a privileged account (with the DEFAULT\_ADMIN\_ROLE role) that play a critical role in governing and regulating the vault-wide operations (e.g., configure parameters, assign roles, and execute privileged operations). It also has the privilege to control or govern the flow of assets managed by this protocol. Our analysis shows that the privileged account needs to be scrutinized. In the following, we examine the privileged account and their related privileged accesses in current contracts.

```

134     function _authorizeUpgrade(address) internal override onlyRole(DEFAULT_ADMIN_ROLE)
         {}

136     /* ----- ADMINISTRATIVE FUNCTIONS ----- */

138     /**
139      * @dev Pause the vault, disabling user interactions.
140      */
141     function pause() external onlyRole(DEFAULT_ADMIN_ROLE) {...}

```

```

143  /**
144   * @dev Unpause the vault, re-enabling user interactions.
145   */
146  function unpause() external onlyRole(DEFAULT_ADMIN_ROLE) {...}

148  /**
149   * @dev Updates the UTC day windows for Open and Processing stages.
150   * @notice Effective immediately. SHOULD be announced to users in advance.
151   */
152  function setStageDays(uint64[2] calldata stageDays_) external override onlyRole(
    DEFAULT_ADMIN_ROLE) {...}

154  /**
155   * @dev Updates the fee recipient address.
156   */
157  function setFeeRecipient(address feeRecipient_) external onlyRole(DEFAULT_ADMIN_ROLE
    ) {...}

159  /**
160   * @dev Updates the minimum deposit amount in asset units.
161   */
162  function setMinDeposit(uint256 minDeposit_) external override onlyRole(
    DEFAULT_ADMIN_ROLE) {...}

164  /**
165   * @dev Updates the minimum redemption amount in share units.
166   */
167  function setMinRedemption(uint256 minRedemption_) external override onlyRole(
    DEFAULT_ADMIN_ROLE) {...}

169  /**
170   * @dev Updates the maximum accepted managed assets. A value of 0 means uncapped.
171   */
172  function setAssetCap(uint256 assetCap_) external override onlyRole(
    DEFAULT_ADMIN_ROLE) {...}

174  /**
175   * @dev Updates the performance fee ratio charged on gains above the high-water mark
176   *
177   */
177  function setPerformanceFeeRatio(uint256 performanceFeeRatio_) external override
    onlyRole(DEFAULT_ADMIN_ROLE) {...}

179  /**
180   * @dev Updates the redemption fee ratio.
181   */
182  function setRedemptionFeeRatio(uint256 redemptionFeeRatio_) external override
    onlyRole(DEFAULT_ADMIN_ROLE) {...}
183  ...
184  function rescue(address token, address to, uint256 amount) external onlyRole(
    DEFAULT_ADMIN_ROLE) {...}

```

Listing 3.3: Example Privileged Operations in vault

We understand the need of the privileged function for contract maintenance, but at the same time the extra power to the privileged account may also be a counter-party risk to the protocol users. It is worrisome if the privileged account is plain EOA account. Note that a multi-sig account could greatly alleviate this concern, though it is still far from perfect. Specifically, a better approach is to eliminate the administration key concern by transferring the role to a community-governed DAO.

In the meantime, the audited contract makes use of the proxy contract to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

**Recommendation** Promptly transfer the privileged account to the intended DAO-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

**Status** This issue has been mitigated with the use of a multi-sig for the privileged account.



## 4 | Conclusion

In this audit, we have analyzed the design and implementation of the `AllDeFi Vault` protocol. `AllDeFi` is a vault infrastructure protocol for professionally managed `DeFi` strategies with institutional custody. It offers a structured vault framework for users to access curated yield opportunities. The audited `vault` facilitates user investment through an epoch-based structure, with deposits processed in a controlled and secure manner. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



## References

- [1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. <https://cwe.mitre.org/data/definitions/1126.html>.
- [2] MITRE. CWE-287: Improper Authentication. <https://cwe.mitre.org/data/definitions/287.html>.
- [3] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
- [4] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [5] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [6] OWASP. Risk Rating Methodology. [https://www.owasp.org/index.php/OWASP\\_Risk\\_Rating\\_Methodology](https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology).
- [7] PeckShield. PeckShield Inc. <https://www.peckshield.com>.